

Product Architecture Design Specification

A simple, case-centered and Python-first blueprint for a high-traffic support product.

The AI is replaceable. The case record, knowledge history and audit trail are the product memory.

PURPOSE	SCOPE	STATUS
Turn the current prototype into an operable product architecture.	Customer side, support console, operations dashboard and shared platform.	Execution draft Version 1.0

01 / DESIGN SPEC

The design starts with six principles

These rules shape the product independently of the current prototype, model provider or cloud vendor.

A conversation is temporary. A case is durable. An answer is useful only when its source and effect can be traced.

1. Case record first

Each individual can own several cases. Status belongs to a case, never to the person as a whole. Every transition is stored as an event.

2. Knowledge is published

Documents are versioned, reviewed and activated. Answers retrieve only from the active version set, while older answers retain their original references.

3. AI reasons; software decides

The model may classify, extract and draft. Authorisation, case transitions, document publication and audit records remain deterministic code paths.

4. Trace every important action

Store who acted, what changed, which sources were retrieved, which model ran, latency, outcome and error identifiers.

5. Stateless compute scales

API containers hold no durable state. Add replicas behind the edge while PostgreSQL remains the source of truth.

6. Split only under pressure

Start as a modular monolith with one worker. Separate services only when ownership, load or failure isolation has measurable value.

Boundary

The product can explain official procedures and manage support work. It must not invent payment confirmation, ownership status, NIMC availability or any other authoritative backend fact. Those claims require a verified system integration or a human decision.

02 / DESIGN SPEC

Three product surfaces, one shared truth

The interfaces are different views over the same people, cases, documents and events. No surface maintains a private copy of business state.

Customer side

- Ask questions and receive source-grounded answers.
- Open a support case without repeating known facts.
- View case status, messages and requested actions.
- Upload supporting documents and give feedback.

Support console

- Search a person, then view all of their cases.
- Filter queues, assign work and change case state.
- Add notes, request information and record decisions.
- Upload, test, publish and roll back documents.

Operations dashboard

- See traffic, errors, latency, queues and uptime.
- Inspect model and retrieval performance.
- Track document ingestion and failed jobs.
- Alert, diagnose, recover and audit incidents.

Shared operating rules

IDENTITY	One person record can have multiple cases and conversations. Identity matching is explicit and auditable.
CASE OWNERSHIP	The case stores status, priority, category, assignee and timestamps. State changes pass through named commands.
KNOWLEDGE	Only published document versions are searchable by customers. Support can preview drafts before activation.
HANDOFF	A support agent sees the customer conversation, extracted facts, cited sources and unresolved questions in one timeline.
AUDIT	Customer, support, system and AI actions are distinguishable. Sensitive fields are masked in operational views.

Primary experience test

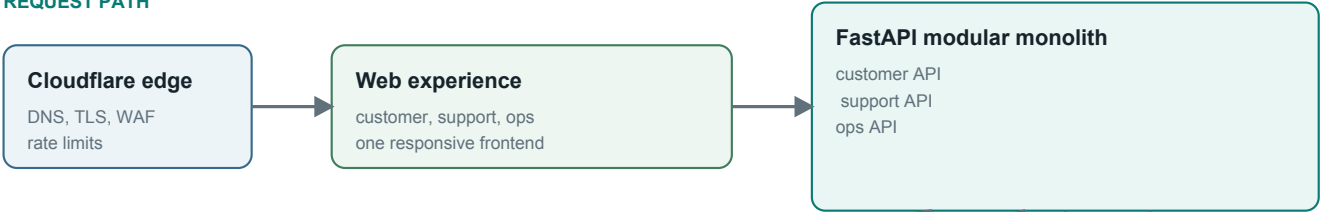
A customer should never need to understand the architecture. Support should never need direct database access. Operations should be able to locate a failing request from a visible error ID.

03 / DESIGN SPEC

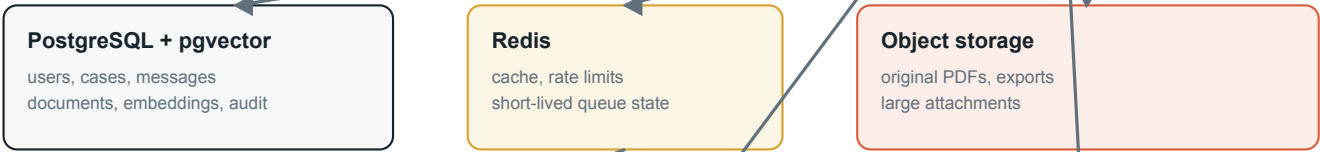
The simplest production architecture

One Python application serves the three product surfaces. A separate Python worker handles slow jobs. Managed services carry durable state and infrastructure burden.

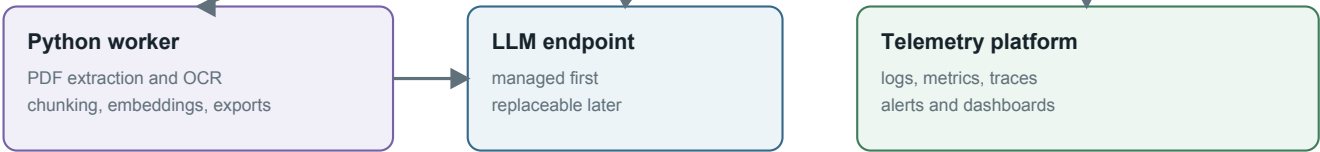
REQUEST PATH



DURABLE SYSTEMS



ASYNCHRONOUS AND EXTERNAL



Deploy the API as multiple identical containers. Only PostgreSQL holds durable application state.

Inside the FastAPI application
Customer, support, operations, cases, knowledge, retrieval, inference and audit are modules with explicit interfaces. They share one deployment but not tangled responsibilities.

Outside the application
PostgreSQL, Redis, object storage, the model endpoint and telemetry are replaceable managed dependencies. The application talks to each through a narrow adapter.

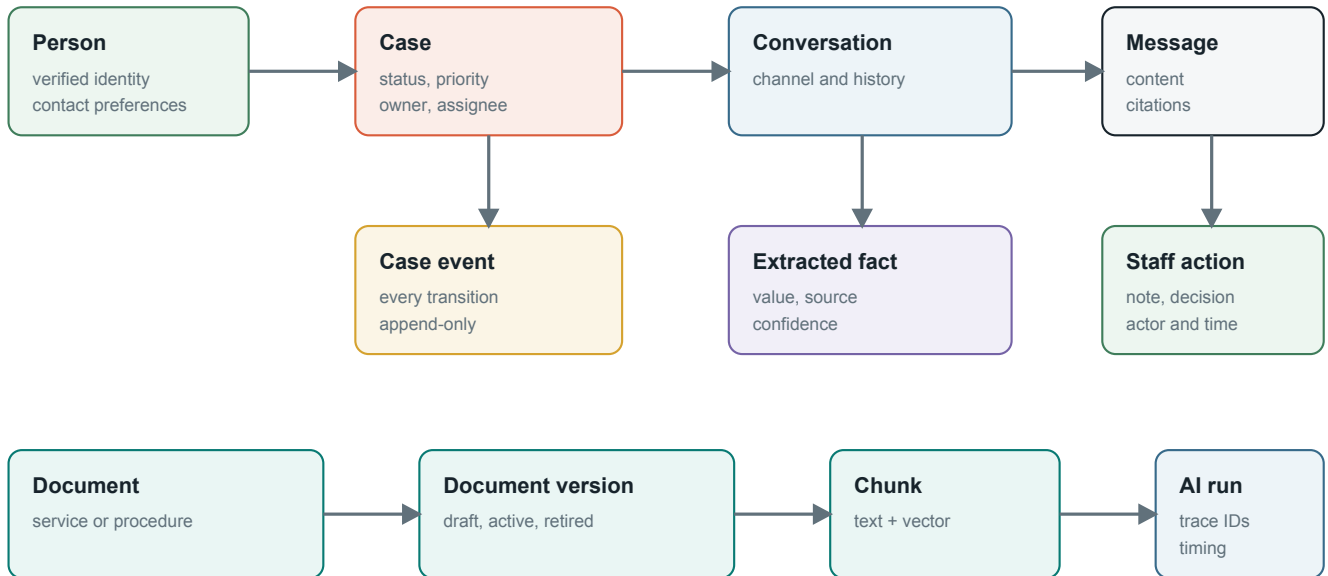
Recommended Python stack

API	FastAPI + Pydantic	Data	PostgreSQL + SQLAlchemy + Alembic
Workers	Celery or Dramatiq + Redis	Vectors	pgvector first
Files	S3-compatible object storage	Telemetry	OpenTelemetry + managed dashboards
UI	Server templates + HTMX, or a thin React client	Delivery	Docker containers + managed platform

04 / DESIGN SPEC

Durable records and case states per individual

The database carries business truth. Model memory is treated as a convenience, never as the record of a customer, case or decision.



Default case lifecycle



Reopening creates a new event; it never erases the earlier resolution.

Case transition rules

- Only named commands may change state.
- Role checks run before every transition.
- Reason, actor and timestamp are mandatory.
- Notifications happen after the event is committed.

AI trace record

- Model and prompt template versions.
- Retrieved document and chunk identifiers.
- Input and output token counts and timing.
- Validation result, fallback and user feedback.

Identity rule

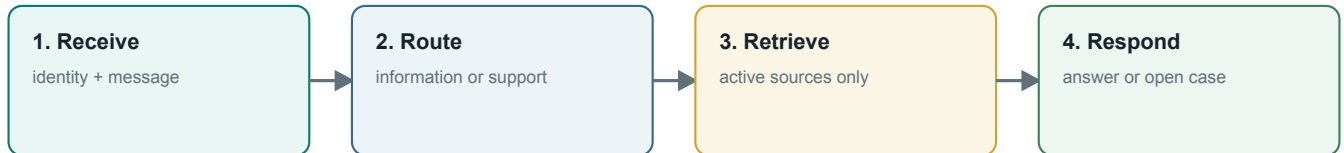
A person can have multiple open and closed cases. Searching a person returns a case list; opening one case shows only its own state and timeline. This prevents one issue from overwriting another and preserves a clean support history.

05 / DESIGN SPEC

Four workflows define the product

Each flow produces durable events and visible outcomes. Slow document work is moved to the worker so customer requests remain responsive.

Customer question



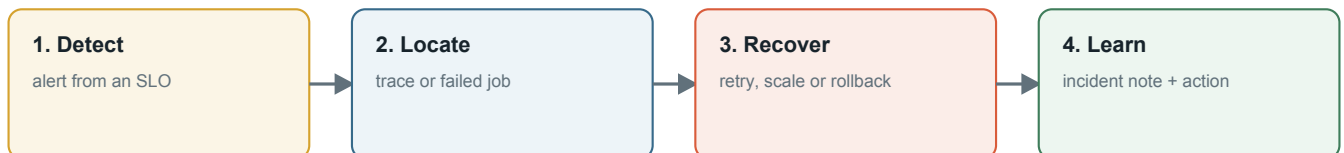
Document publication



Support case handling



Operations incident

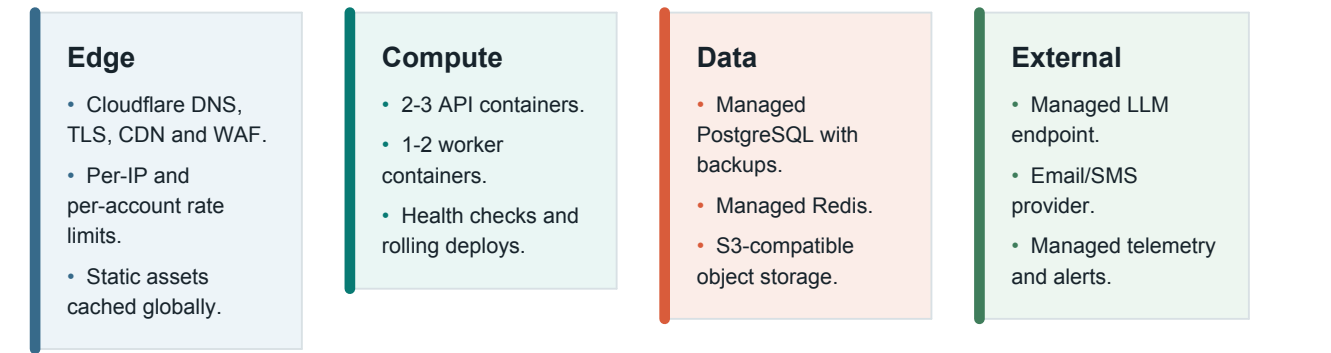


Publishing guarantee: a document version is either entirely active or entirely inactive. Partial indexes are never visible to customers.

06 / DESIGN SPEC

Hosted as managed containers and managed data

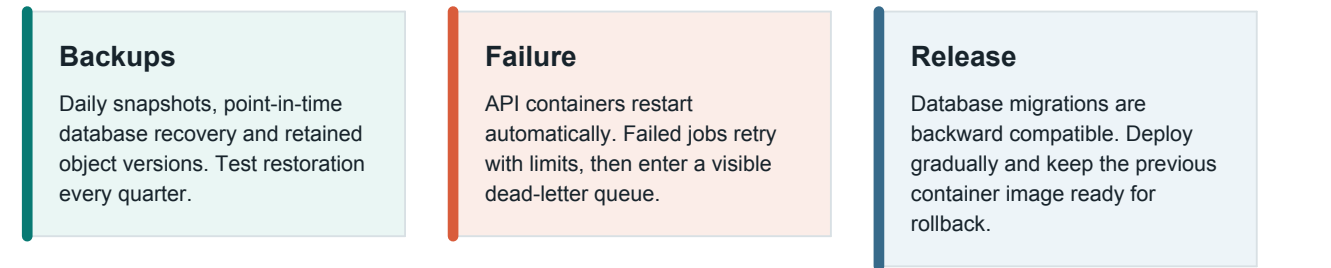
The deployment starts with low operational burden and keeps an exit path to a larger container platform without rewriting the application.



Recommended deployment path

STAGE	PLATFORM SHAPE	MOVE WHEN
Launch	Render, Fly.io or an equivalent managed container platform. One web service, one worker service and managed data.	Use this while the team is small and traffic is uncertain.
Scale	AWS ECS Fargate or an equivalent managed container service, with managed PostgreSQL, Redis and object storage.	Move for stronger compliance, custom networking or sustained high traffic.
Specialise	Separate inference, ingestion or retrieval only where load and ownership justify independent scaling.	Split a module when it has a measured bottleneck or distinct reliability target.

Availability and recovery baseline



07 / DESIGN SPEC

Operations and security are product features

The dashboard is not a wall of technical charts. It answers whether customers are being served, where work is blocked and what changed.

SERVICE HEALTH	CUSTOMER FLOW	AI QUALITY	KNOWLEDGE
Request volume error rate p50/p95 latency uptime	answers delivered cases opened case age resolution time	retrieval hit rate fallback rate user feedback cost per answer	active versions failed ingestion publish age zero-result queries

Initial service targets

99.9% monthly API availability	< 500 ms p95 non-AI API latency	< 12 sec p95 completed AI answer	< 5 min document publish after processing
--	--	---	--

Access and privacy

- Roles: customer, support, supervisor, admin, auditor.
- Encrypt transport and stored data.
- Mask PII in logs and dashboards.
- Set retention and deletion rules by data class.
- Require stronger authentication for staff.

Governance and resilience

- Audit document publication and case decisions.
- Store secrets outside code and images.
- Validate uploads and restrict file types.
- Back up, restore and record recovery drills.
- Rate-limit abuse and preserve request IDs.

Alert only on conditions that require action: sustained error rate, missing telemetry, growing job backlog, failing document ingestion, database pressure, LLM quota exhaustion or a breached case-age target.

08 / DESIGN SPEC

Implementation plan

Build vertical slices in dependency order. Every phase ends in a user-visible capability and a testable operational outcome.

PHASE	BUILD	EXIT CHECK
0 Foundation	Repository layout, environments, container images, CI, secrets, health checks.	API and worker deploy from one commit.
1 Durable core	PostgreSQL schema, migrations, users, cases, events, audit log, role access.	Case survives restart and has a complete timeline.
2 Customer side	Sign-in, chat, case creation, case history, attachments, feedback.	A customer can ask, continue, and track a case.
3 Support console	Queues, filters, assignment, notes, state changes, customer history.	Staff can resolve a case without database access.
4 Knowledge	Upload, version, parse/OCR, chunk, embed, validation, publish, rollback.	A new PDF can be published and cited safely.
5 AI + traceability	Binary routing, grounded generation, retrieval trace, model run record, fallback.	Every answer can be reconstructed from stored metadata.
6 Operations	Metrics, logs, traces, dashboards, alerts, backup checks, runbooks.	An operator can detect and diagnose failure.
7 Launch	Security review, load test, recovery drill, staged rollout, support training.	SLOs pass at 2x expected peak traffic.

Suggested team and timing

One strong Python engineer

Approximately 6-8 weeks for a controlled pilot, provided identity, messaging and model integrations are straightforward.

Two engineers

Approximately 4-6 weeks when backend/platform and interface work progress in parallel with shared review.

Planning condition

These are delivery estimates, not commitments. External approvals, data migration and security review can extend the calendar.

Module layout

app/api	customer, support and operations routes	app/cases	case commands, state and events
app/knowledge	documents, versions and publishing	app/ai	routing, retrieval, inference and validation
app/platform	database, auth, audit and telemetry	worker/tasks	ingestion, embeddings, notifications, exports
migrations	versioned database changes	tests	unit, integration, retrieval and load tests

09 / DESIGN SPEC

Capacity, scaling triggers and launch decisions

Traffic ranges are planning envelopes for the described topology. The LLM endpoint, prompt length and answer time usually set the true ceiling.

DEPLOYMENT	DAILY USERS	MESSAGES / DAY	PEAK / HOUR
Launch	10k - 50k	30k - 150k	500 - 3k
Scaled, same design	100k+	300k - 1m	5k - 30k

Assumed launch shape: 2-3 API containers, 1-2 workers, indexed managed PostgreSQL, managed Redis, object storage and a managed LLM quota sized for the expected peak.

LLM throughput per second = allowed concurrent model calls / average model response time

Example: 20 concurrent calls at 5 seconds each is about 4 requests per second, or 14,400 per hour. If classification and answering each use an LLM call, effective answer capacity can be roughly halved unless the classifier is much faster.

Scale when a measured trigger persists

Add API replicas

- CPU or memory stays above 70%.
- Non-AI p95 latency misses target.
- Connection queues grow.

Add workers

- Job backlog age exceeds target.
- Document publishing slows.
- Retries compete with new work.

Upgrade a dependency

- Database pressure remains after indexing.
- LLM quota or latency is the bottleneck.
- Vector search misses its target.

Keep the first version intentionally small

USE NOW	DEFER UNTIL EVIDENCE REQUIRES IT
FastAPI modular monolith	Separate microservices
PostgreSQL + pgvector	Dedicated vector database
Redis + Python worker	Kafka or a workflow platform
Managed LLM endpoint	Self-hosted GPU fleet
Single region + tested backups	Active-active multi-region

10 / DESIGN SPEC

Definition of done

The architecture is complete when the three product surfaces work together under load, can be operated without direct database access and can explain what happened.

Customer

A user can ask an informational question, receive a concise source-grounded answer, open a case and view its current state and history.

Support

An authorised agent can find a person, choose one of several cases, change state, add a note, request information and resolve the case.

Knowledge

An authorised editor can upload a PDF, review extraction, test retrieval, publish a new version and roll back without partial exposure.

Operations

An operator can see health, traffic, latency, errors, backlog, model behaviour and document failures, then follow a runbook from an alert.

Governance

Every case decision, document publication and AI answer has an actor or system identity, timestamp, source references and request identifier.

Scale

A load test at twice the expected launch peak meets service targets; backup restoration and deployment rollback are demonstrated.

Build the durable case and knowledge systems first. Let the AI improve inside those boundaries without becoming the boundary.

Immediate execution order

1	Create the PostgreSQL schema and migrations for people, cases, case events, conversations, messages and audit events.
2	Move the existing binary router and grounded answer flow behind the new conversation and case application services.
3	Build the support case view and document publishing workflow before adding visual polish to the operations dashboard.
4	Deploy two API containers and one worker, instrument them, then load test the full answer path with realistic prompts and documents.

END OF DESIGN SPEC | Version 1.0